

計算複雑性

今日学ぶこと

- 時間計算量・空間計算量
 - 定義
 - 表記法
 - 計算量の分析
 - 様々な計算量
 - 時間と空間のトレードオフ
- 複雑性クラス
 - オートマトン・チューリングマシン
 - 複雑性クラス
 - P vs NP
 - NP・EXP との付き合い方

計算量

あるアルゴリズムの、入力の長さ N が大きくなるにつれ、
時間(CPU)・空間(メモリ)がどう変化するかを関数で表現する。

	最良時間計算量	平均時間計算量	最悪時間計算量	空間計算量
クイックソート	$O(n)$	$O(n \log(n))$	$O(n^2)$	$O(n)$
マージソート	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
ボゴソート	$O(n)$	$O(n \times n!)$	∞	$O(n)$

ボゴソート: 入力がソートされてるか確認し、違ったらシャッフル。以下繰り返し。

単に計算量というと、平均時間計算量を指し、
空間計算量と言うと、最悪空間計算量を指す。

(最悪の時にメモリが足りないと動かないでは困る)

計算量

```
1 void merge_sort(int n, int *data, int *buffer) {
2     if (n <= 1) {
3         return;
4     }
5     const int m = n/2;
6     merge_sort(m, data, buffer);
7     merge_sort(n-m, data+m, buffer);
8     int t = 0, ai = 0, bi = m;
9     while (ai != m && bi != n) {
10         if (data[ai] < data[bi]) {
11             buffer[t++] = data[ai++];
12         } else {
13             buffer[t++] = data[bi++];
14         }
15     }
16     while (ai != m) {
17         buffer[t++] = data[ai++];
18     }
19     while (bi != n) {
20         buffer[t++] = data[bi++];
21     }
22     for (int i = 0; i < n; i++) {
23         data[i] = buffer[i];
24     }
25 }
```

計算量は $O(n\log(n))$ 。

計算量

```
1 void merge_sort(int n, int *data, int *buffer) {
2     if (n <= 1) {
3         return;
4     }
5     const int m = n/2;
6     merge_sort(m, data, buffer);
7     merge_sort(n-m, data+m, buffer);
8     int t = 0, ai = 0, bi = m;
9     while (ai != m && bi != n) {
10         if (data[ai] < data[bi]) {
11             buffer[t++] = data[ai++];
12         } else {
13             buffer[t++] = data[bi++];
14         }
15     }
16     while (ai != m) {
17         buffer[t++] = data[ai++];
18     }
19     while (bi != n) {
20         buffer[t++] = data[bi++];
21     }
22     for (int i = 0; i < n; i++) {
23         data[i] = buffer[i];
24     }
25 }
```

計算量は $O(n\log(n))$ 。

n はアルゴリズムが想定する入力サイズ。
(ここでは配列の要素数)

n はアルゴリズムが想定する入力サイズ。
配列の各要素のバイト数 b など、
アルゴリズムの想定しないものは含まない。

(b に比例する時間が必要になる)

計算量

```
1 void merge_sort(int n, int *data, int *buffer) {
2     if (n <= 1) {
3         return;
4     }
5     const int m = n/2;
6     merge_sort(m, data, buffer);
7     merge_sort(n-m, data+m, buffer);
8     int t = 0, ai = 0, bi = m;
9     while (ai != m && bi != n) {
10         if (data[ai] < data[bi]) {
11             buffer[t++] = data[ai++];
12         } else {
13             buffer[t++] = data[bi++];
14         }
15     }
16     while (ai != m) {
17         buffer[t++] = data[ai++];
18     }
19     while (bi != n) {
20         buffer[t++] = data[bi++];
21     }
22     for (int i = 0; i < n; i++) {
23         data[i] = buffer[i];
24     }
25 }
```

計算量は $O(n\log(n))$ 。

n はアルゴリズムが想定する入力サイズ。
(ここでは配列の要素数)

n はアルゴリズムが想定する入力サイズ。
配列の各要素のバイト数 b など、
アルゴリズムの想定しないものは含まない。

(b に比例する時間が必要になる)

分析の際には n が増えた際に
最も計算が必要になる部分のみを考慮する。

計算量

```
1 void merge_sort(int n, int *data, int *buffer) {
2     if (n <= 1) {
3         return;
4     }
5     const int m = n/2;
6     merge_sort(m, data, buffer);
7     merge_sort(n-m, data+m, buffer);
8     int t = 0, ai = 0, bi = m;
9     while (ai != m && bi != n) {
10         if (data[ai] < data[bi]) {
11             buffer[t++] = data[ai++];
12         } else {
13             buffer[t++] = data[bi++];
14         }
15     }
16     while (ai != m) {
17         buffer[t++] = data[ai++];
18     }
19     while (bi != n) {
20         buffer[t++] = data[bi++];
21     }
22     for (int i = 0; i < n; i++) {
23         data[i] = buffer[i];
24     }
25 }
```

計算量は $O(n\log(n))$ 。

n はアルゴリズムが想定する入力
(ここでは配列の要素数)のサイズ。

n はアルゴリズムが想定する入力
のサイズ。
配列の各要素のバイト数 b など、
アルゴリズムの想定しないものは含まない。

(bに比例する時間が必要になる)

分析の際にはnが増えた際に
最も計算が必要になる部分のみを考慮する。

計算量

```
1 void merge_sort(int n, int *data, int *buffer) {
2     if (n <= 1) {
3         return;
4     }
5     const int m = n/2;
6     merge_sort(m, data, buffer);
7     merge_sort(n-m, data+m, buffer);
8     int t = 0, ai = 0, bi = m;
9     while (ai != m && bi != n) {
10         if (data[ai] < data[bi]) {
11             buffer[t++] = data[ai++];
12         } else {
13             buffer[t++] = data[bi++];
14         }
15     }
16     while (ai != m) {
17         buffer[t++] = data[ai++];
18     }
19     while (bi != n) {
20         buffer[t++] = data[bi++];
21     }
22     for (int i = 0; i < n; i++) {
23         data[i] = buffer[i];
24     }
25 }
```

計算量は $O(n\log(n))$ 。

n はアルゴリズムが想定する入力サイズ。
(ここでは配列の要素数)

n はアルゴリズムが想定する入力サイズ。
配列の各要素のバイト数 b など、
アルゴリズムの想定しないものは含まない。

(bに比例する時間が必要になる)

分析の際にはnが増えた際に
最も計算が必要になる部分のみを考慮する。

表記法

計算機科学的な定義

O	ビッグオー	上界(以下)
Θ	ビッグシータ	上界かつ下界 (同じ)
Ω	ビッグオメガ	下界(以上)
o	リトルオー	上界(より小さい)
ω	リトルオメガ	下界(より大きい)

n^2 に対して、

	(n)	(n^2)	(n^3)
O	$\times$	$\bigcirc$	$\bigcirc$
Θ	$\times$	$\bigcirc$	$\times$
Ω	$\bigcirc$	$\bigcirc$	$\times$
o	$\times$	$\times$	$\bigcirc$
ω	$\bigcirc$	$\times$	$\times$

$f(n)$ が $O(g(n))$ であるとは、

ある c が与えられた時、 $n > lb$ となる全ての n について、 $f(n) \leq c \times g(n)$ となるような lb が存在している。

ということを意味する。

厳密に使い分けが必要な論文等を除いて、
 o しかほぼ使われず、しかも Θ の意味で普段使いされる。

(n^2 は $O(n^3)$ です。と言うと、何だこいつ..扱いされる)

計算量の分析

マージソート

長さ n の配列のソートに必要な時間を $f(n)$ とすると、

$$\begin{aligned} f(n) &= c \times n + 2 \times f\left(\frac{n}{2}\right) \\ &= c \times n + 2 \times \left(c \times \frac{n}{2} + 2 \times f\left(\frac{n}{2^2}\right) \right) \\ &= c \times n + 2 \times \left(c \times \frac{n}{2} + 2 \times \left(c \times \frac{n}{2^2} + 2 \times f\left(\frac{n}{2^3}\right) \right) \right) \\ &= \dots \\ &= \overbrace{c \times n + c \times n + \dots + c \times n}^{\log_2 n \text{個}} + 2^{\log_2 n} f(1) \\ &= O(n \log_2 n) + O(n) \\ &= O(n \log_2 n) \end{aligned}$$

計算量の分析

シュトラッセンのアルゴリズム

$(n \times n)$ 行列同士の掛け算を、 $(\frac{n}{2} \times \frac{n}{2})$ 行列同士の掛け算7回とその和で計算する。

$(n \times n)$ 行列同士の掛け算に必要な算時間を $f(n)$ とすると、

$$\begin{aligned}
 f(n) &= c \times n^2 + 7 \times f\left(\frac{n}{2}\right) \\
 &= c \times n^2 + 7 \times \left(c \times \left(\frac{n}{2}\right)^2 + 7 \times f\left(\frac{n}{2^2}\right)\right) \\
 &= c \times n^2 + 7 \times \left(c \times \left(\frac{n}{2}\right)^2 + 7 \times \left(c \times \left(\frac{n}{2^2}\right)^2 + 7 \times f\left(\frac{n}{2^3}\right)\right)\right) = \dots \\
 &= c \times n^2 \times \left(\frac{7}{4} + \left(\frac{7}{4}\right)^2 + \dots + \left(\frac{7}{4}\right)^{\log_2(n)}\right) + 7^{\log_2(n)} f(1) \\
 &= c \times n^2 \times \frac{\frac{7}{4} \left(\frac{7}{4}\right)^{\log_2(n)} - 1}{\frac{7}{4} - 1} + 7^{\log_2(n)} f(1) \\
 &= O\left(n^2 \left(\frac{7}{4}\right)^{\log_2(n)}\right) + O(7^{\log_2(n)}) \\
 &= O(7^{\log_2(n)}) + O(7^{\log_2(n)}) = O(7^{\log_2(n)}) = O(n^{\log_2(7)})
 \end{aligned}$$

計算量の分析

数式の補足

$$\begin{aligned}
 (a \times b)^c &= \overbrace{(a \times b) \times (a \times b) \times \cdots \times (a \times b)}^{c \text{ 個}} \\
 &= \overbrace{a \times a \times \cdots \times a}^{c \text{ 個}} \times \overbrace{b \times b \times \cdots \times b}^{c \text{ 個}} \\
 &= a^c \times b^c
 \end{aligned}$$

$$\log_a b = c \iff a^c = b$$

$$a^{\log_a b} = b$$

$$\begin{aligned}
 a^{\log_b c} &= (c^{\log_c a})^{\log_b c \text{ 個}} \\
 &= \overbrace{c \times c \times \cdots \times c}^{\log_b c \times \log_c a = \log_b a \text{ 個}} \\
 &= c^{\log_b a}
 \end{aligned}$$

$$\log_a b \times \log_b c = \log_a (a^{\log_a b \times \log_b c})$$

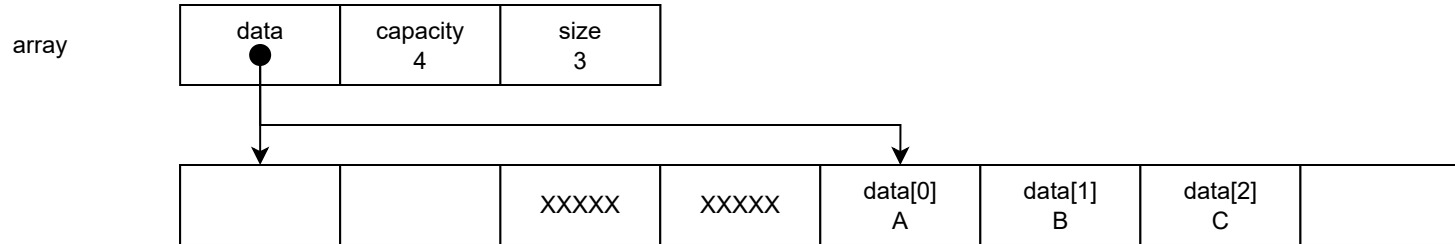
$$\begin{aligned}
 &= \log_a \overbrace{(\overbrace{a \times a \times \cdots \times a}^{\log_a b \text{ 個}}) \times \cdots \times (\overbrace{a \times a \times \cdots \times a}^{\log_a b \text{ 個}})}^{\log_b c \text{ 個}} \\
 &= \log_a \overbrace{b \times b \times \cdots \times b}^{\log_b c \text{ 個}} = \log_a c
 \end{aligned}$$

$$\begin{aligned}
 1 + x + \cdots + x^n &= \frac{(1-x)(1+x+\cdots+x^n)}{1-x} \\
 &= \frac{(1+x+\cdots+x^n) - (x+x^2+\cdots+x^{n+1})}{(1-x)} \\
 &= \frac{1-x^{n+1}}{1-x}
 \end{aligned}$$

$$\begin{aligned}
 n^2 \times \left(\frac{7}{4}\right)^{\log_2(n)} &= n^2 \times \left(\frac{1}{2} \times \frac{1}{2} \times 7\right)^{\log_2(n)} \\
 &= n^2 \times \frac{1}{2^{\log_2(n)}} \times \frac{1}{2^{\log_2(n)}} \times 7^{\log_2(n)} \\
 &= n^2 \times \frac{1}{n} \times \frac{1}{n} \times n^{\log_2 7} \\
 &= n^{\log_2 7}
 \end{aligned}$$

計算量の分析

動的配列への挿入



$2^n + 1$ 回目のデータ追加の際に、 $c \times 2^n$ のデータ移動コストが必要。

2^n 回のデータ追加にかかるコスト

= 単純追加のコスト $\times 2^n$ + データ移動コスト

= $O(2^n) + c \times (1 + 2 + \dots + 2^{n-1})$

= $O(2^n) + c \times (2^n - 1)$

= $O(2^n)$

よって1回当たりに均した計算量は $O(1)$ (償却定数時間)

様々な計算量

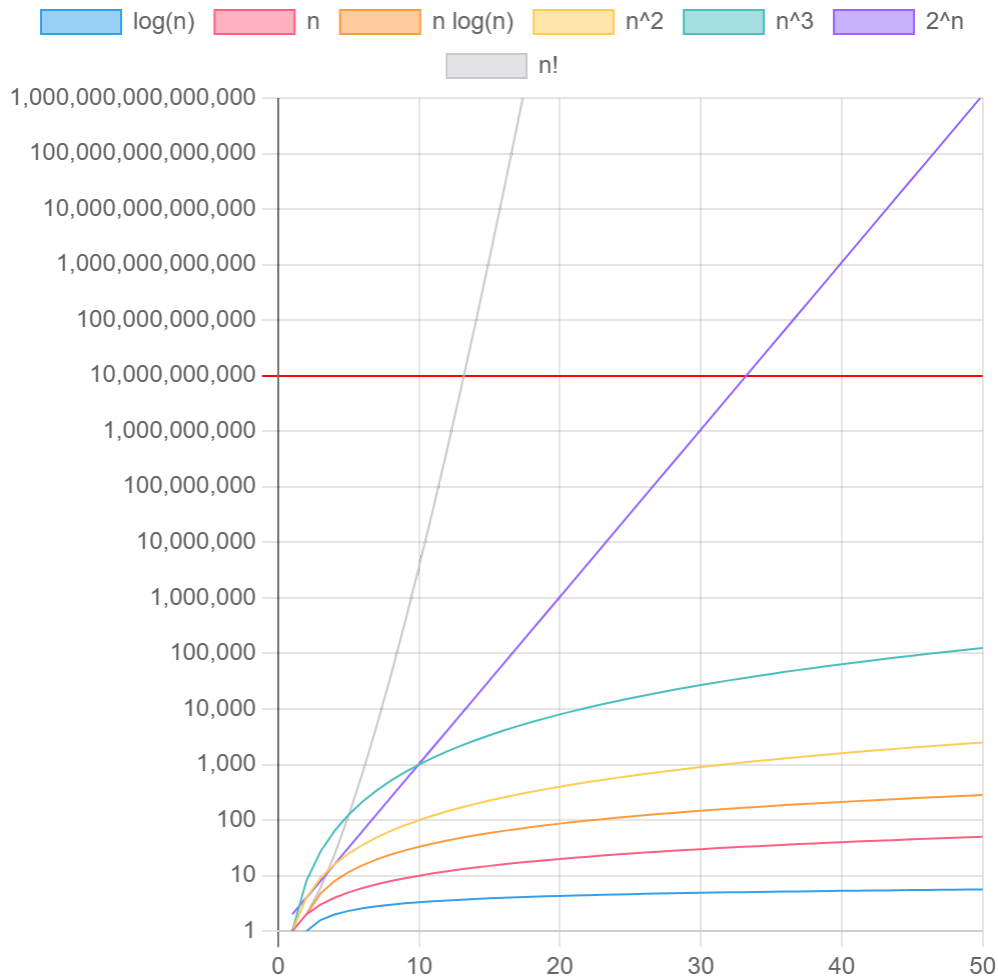
$O(1)$	定数時間	配列へのアクセス
$O(\log_2 n)$	対数時間	二分探索
$O(n)$	線形時間	線形探索
$O(n \log_2 n)$	準線形時間 対数線形時間	クイックソート・マージソート
$O(n^2)$	二乗時間 平方時間	挿入ソート
$O(n^c)$	(これより上も全部含め) 多項式時間	行列積・ <u>ガウスの消去法</u> ・素数判定
$O(2^n)$	指数時間	巡回セールスマン (TSP)・ナップザック 充足可能性 (SAT)・全部分集合
$O(n!)$	階乗時間	<u>ハミルトン閉路問題</u> ・全順列

様々な計算量

	n=10	n=20	n=30	n=100	n=1000	n=1000000
$O(1)$	1	1	1	1	1	1
$O(\log_2 n)$	2.3	4.3	4.9	6.6	10	20
$O(n)$	10	20	30	100	1000	1E6
$O(n \log_2 n)$	23	86	147	664	9955	2.0E7
$O(n^2)$	100	400	8000	10000	1E6	1E12
$O(n^3)$	1000	8000	27000	1E6	1E9	1E18
$O(2^n)$	1024	1.0E6	1.1E9	1.3E30	1.1E301	9.9E301029
$O(n!)$	3.6E6	2.4E18	2.7E32	9.3E157	4.0E2567	8.3E5565708

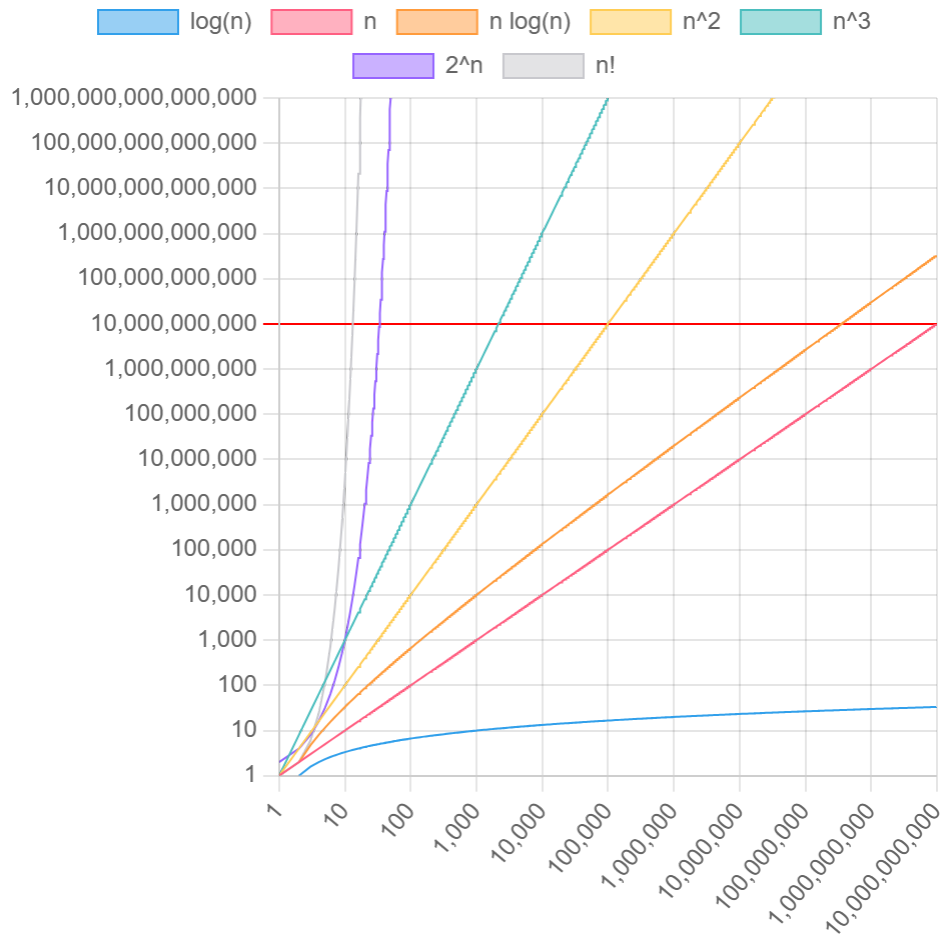
最近の CPU の周波数は 3GHz = 1秒に3E9回計算を進められる。

様々な計算量

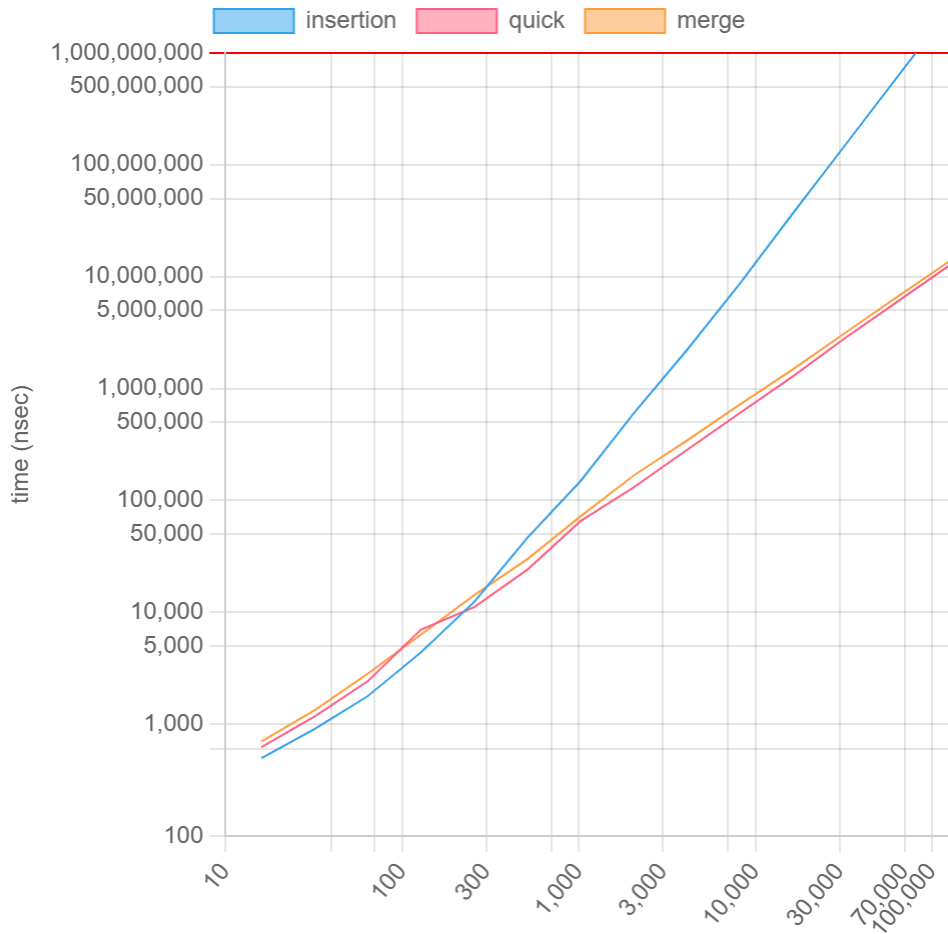


$y = 10^{10} = 100$ 億は、CPUで少なくとも秒単位で時間がかかる

$n!$ は $n=14$, 2^n は 34 , n^3 は 2155 , n^2 は 10 万, $n \log n$ は 3.6 億でそれより遅くなる。



性能の計測



Nが十分大きいとき、
挿入ソートはNが2倍で、必要時間が4倍、
クイック・マージソートは2倍強。

```
void insertion_sort(int n, int *data) {
    for (int s = 1; s < n; s++) { // s: sorted length
        int t = 0;
        int d = data[s];
        for (; t < s && data[t] <= d; t++) ;
        for (int i = s-1; i >= t; i--)
            data[i+1] = data[i];
        data[t] = d;
    }
}
```

```
void quick_sort(int n, int *data) {
    if (n <= 1) {
        return;
    }
    int s = 0, e = n;
    int t = 0;
    int pivot = data[0];
    while (t < e) {
        if (data[t] < pivot) {
            swap(data, s++, t++);
        } else if (data[t] == pivot) {
            t++;
        } else {
            swap(data, t, --e);
        }
    }
    quick_sort(s, data);
    quick_sort(n-e, data+e);
}
```

性能計測は様々なノイズの影響を受ける。

CPUキャッシュ・目的コード以外のオーバーヘッドなど。

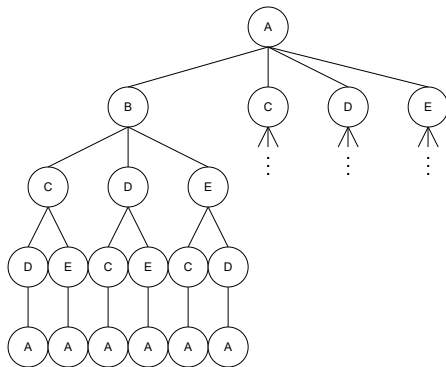
CPU周波数の変動機能や、他プロセスの影響は可能な限り排除したい。

時間と空間のトレードオフ

問題により、多く空間を使うと時間を節約できるアルゴリズムがある。

巡回セールスマン問題

総当たり法



全ての可能性について総当たりを行う

時間計算量は $O(n!)$

空間計算量は $O(n^2)$

(全枝を保持するコスト)

動的計画法

(既に訪れた点の集合 x 最後に訪れた点)
となる最短距離のテーブルを保持し、

$$TBL[V', x] (x \in V')$$

$$= \min_{\forall v \in V' - \{x\}} (TBL[V' - \{x\}, v] + E[v, x])$$

と計算する。

n 点通って x にいる状態の最短距離は、

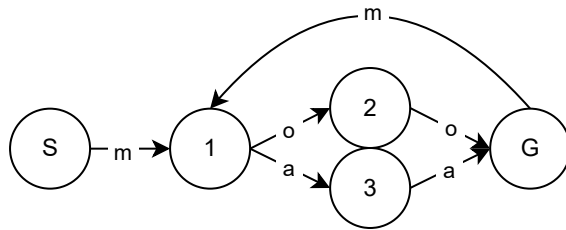
x 以外の $n-1$ 点通った各最短距離から計算できる、という意味の式

時間計算量は $O(n^2 2^n)$

空間計算量は $O(n 2^n)$

複雑性クラス

オートマトン



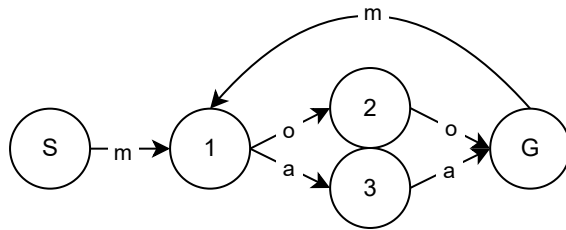
状態	入力	遷移先
S	m	1
1	o	2
1	a	3
2	o	G
3	a	G
G	m	1

上記は $(moo|maa)^+$ の正規表現

オートマトンの種類

決定性	状態・入力に対し遷移が最大1つ (現在状態は高々1つに決定)
非決定性	状態・入力に対し遷移は複数可 (現在の状態が複数あり得る)
有限	状態の数が有限

オートマトン

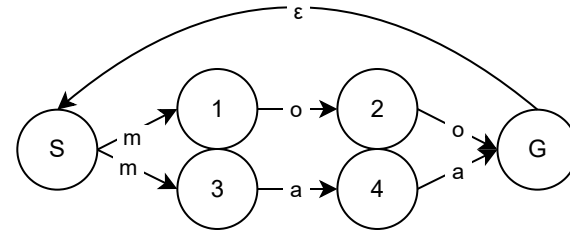


状態	入力	遷移先
S	m	1
1	o	2
1	a	3
2	o	G
3	a	G
G	m	1

上記は $(moo|maa)^+$ の正規表現

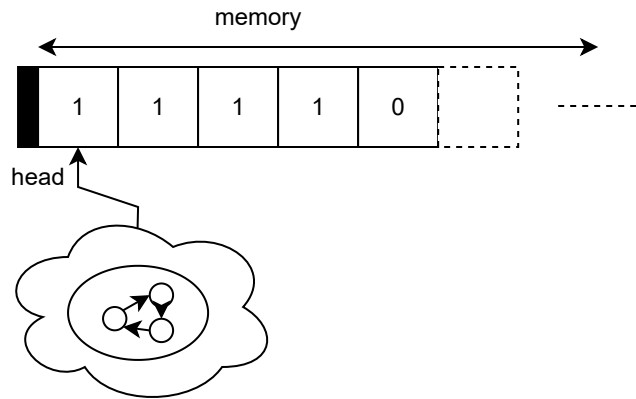
オートマトンの種類

決定性	状態・入力に対し遷移が最大1つ (現在状態は高々1つに決定)
非決定性	状態・入力に対し遷移は複数可 (現在の状態が複数あり得る)
有限	状態の数が有限



非決定性有限オートマトン版。εはいつでも遷移して良い

チューリングマシン



状態	メモリ	状態遷移	メモリ書込	ヘッド位置
S	1	a	-	→
a	1	S	-	→
a	0	G	-	-

(入力に並んだ1の数が奇数個か判定するプログラム)

決定性有限オートマトンと、
無限のメモリを組み合わせたもの。
メモリはヘッドの指す位置だけ読み書きできる。

状態と、ヘッドの指すメモリに応じ、
状態を遷移し、メモリを書き、ヘッドを動かす。

非決定性チューリングマシン

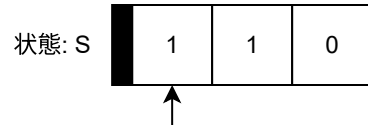
状態	メモリ	状態遷移	メモリ書込	ヘッド位置
S	-(何でも良い)	S	0	→
S	1	a	-	→
a	1	G	-	-

チューリングマシンの遷移が
非決定性になったもの。

(気持ち的には、複数の世界線を同時に計算できる、

不思議魔法コンピュータ。)

(入力に 11 が含まれていれば、それより前を 0 に書き換えて正常終了するプログラム)



複雑性クラス

P チューリングマシン^(TM. この文脈では普通のコンピュータと見做してOK)を使って、
 n の多項式 (n^x) 時間で解ける問題の集合
 ソート・最短距離・行列乗算・素数判定など

NP 非決定性チューリングマシン^(不思議コンピュータ)を使って、
 n の多項式 (n^x) 時間で解ける問題の集合
 = 答えが YES となる際に、証拠が与えられると、
 普通のチューリングマシンで P で検算することが出来る問題

(世界線の分岐の際にどの選択肢を取れば G に辿り着くか? も証拠の一つ)

巡回セールスマン問題・ナップザック問題

(答えを x 以下にできますか? という形にすると、YES なら証拠を出せる)

素因数分解・充足可能性判定(SAT)など ^(NP完全問題)

EXPTIME TMを使って、 n の指数時間 (x^n) で解ける問題の集合

PSPACE TM を使って、 n の多項式 (n^x) のメモリを使用して解ける問題
 倉庫番・五目並べなど ^(PSPACE完全問題)

$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$ かつ、 $P \subsetneq EXPTIME$

P VS NP

$P = NP$ なのか、 $P \neq NP$ なのかは未解決問題。(科学者は $P \neq NP$ と考えている)

$P = NP$ なら → 素因数分解が $P \rightarrow$ RSA 暗号の解読に効率的(?)な方法が存在

NP完全問題

NP問題で、これを多項式 (n^x) 回解くことで任意の NP 問題を解くことができるもの。
NP完全問題が1つでも P で解ければ、全 NP 問題が P で解けて $P = NP$ となる。

(NPの中で一番難しい問題。NP完全以上に難しいことをNP困難と言うが、NP完全の意味で使われがち)



充足可能性問題 (SAT)

論理変数 $x_1, x_2, \dots$ で構成した論理式
を真にする変数割当が存在するか?

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2) \wedge x_2$$

答え: $x_1 = F, x_2 = T, x_3 = T$ で可能

クックの定理

SAT を多項式回解くと、
任意の NP の問題が解ける。
SAT は NP 完全。

(気持的には、各時間に取り得るメモリや状態を論理式で表現して、

SATを解く = 非決定性チューリングマシンをシミュレートする となる状態を作る)

NP / EXP に出会ったら

学術上は $P \neq NP$ とは証明されていないが、実務上指数時間扱いして良い。

NP完全問題や、NP であるが、NP 完全かどうか不明な問題(素因数分解・グラフ同型問題)に出会ったら、正攻法で大きな N を計算しようとしてはいけない。

- 十分な N が小さい時だけ解くので問題が無いか考える
- N が大きくなった際に、一定時間以内に答えが見つからなかったら諦めて良いか考える
- 近似アルゴリズムを適用できないか考える
- SAT Solver を試してみる
 - SAT Competition 2024 は kissat が優勝

複雑クラス (空間)

DSPACE($f(n)$) チューリングマシンを使って、
多項式メモリ $O(f(n))$ を使って解ける問題
DSPACE を全部の多項式について集めると PSPACE

NSPACE($f(n)$) 非決定性チューリングマシンを使って、
多項式メモリ $O(f(n))$ を使って解ける問題
NSPACE を全部の多項式について集めると NPSPACE

サヴィッチの定理

$$NPSPACE(f(n)) \subseteq DSPACE(f^2(n))$$

$$PSPACE = NPSPACE$$

非決定性チューリングマシンでメモリ x で問題が解けるとき、
決定性チューリングマシンでは、メモリ x^2 あれば解ける